

BeatHub Database Architecture & Analytics Blueprint

Prepared for production-scale growth from 1M to 5M users. This blueprint is anchored to the current BeatHub codebase and extends it with a scalable data strategy for hot writes, analytical reads, and predictable index maintenance.

Scope note: the repository currently exposes a small analytics surface around `/api/analytics/top-users`. The blueprint below generalizes that foundation into a production-grade analytics and indexing model, including a proposed `listening_history` fact collection that is not yet present in the codebase but is required for the identified scale issues.

1. Query Audit & Analysis

The audit groups the dominant workload into a small number of reusable access patterns. Each pattern is described by its target collection, the effective filter/sort shape, and the workload class that should drive index design.

Query pattern	Collection(s)	Query shape	Category	Operational goal
Trending Feed / Homepage	listening_history, songs	playedAt >= last 30 days; group by songId; sort plays desc; limit top N	High-frequency read, aggregation-heavy	Minimize latency for the most visible landing page.
Artist Analytics	listening_history, songs	artistId = ?; playedAt range; group by day/song	Analytical read	Support dashboards without scanning the full event store.
Song Search / Lookup	songs, artists	title prefix or exact title; optional artist filter; sort by relevance/name	High-frequency read	Keep catalog browsing responsive and predictable.
User Profile / Library	users, songs	_id = ?; populate likedSongs; sort related content by recency	Interactive read	Serve profile and preference views with low tail latency.
Playlist Detail / Owner View	playlists, songs	user = ?; sort playlists by newest; join song details	High-frequency read	Support authenticated browsing and curation workflows.
Listening History Ingest	listening_history	insertOne / insertMany with minimal secondary lookups	Write-heavy hot path	Absorb high-volume play events without write amplification.

Shared index families outperform per-query indexing because BeatHub's workload is pattern-based. Many endpoints differ only by one predicate or by a narrower time window, so one compound index can satisfy several queries with the same leading equality key and sort order. This lowers write amplification, reduces memory pressure from extra B-trees, and keeps the planner's choice set smaller. In practice, a single user-oriented or time-oriented compound index usually benefits many screens, whereas one-off indexes add maintenance cost without materially improving latency.

2. Formal Indexing Plan

The plan below assumes MongoDB's ESR rule: Equality predicates should lead, Sort fields should follow, and Range predicates should appear after equality keys where possible. Selectivity is called out explicitly because a theoretically correct index can still be poor if it starts with a low-cardinality field that does not narrow the working set enough.

Collection	Recommended index	ESR placement	Cardinality / selectivity	Why this index exists
songs	{ artist: 1, _id: -1 }	E then S	artist is moderately high-cardinality; _id is unique and stable for pagination	Supports artist pages, artist analytics joins, and infinite scroll by recency.
songs	{ title: 1 }	E	title has decent selectivity for exact or prefix lookups, but not for fuzzy search	Covers catalog lookup without forcing a full collection scan.
users	{ email: 1 } unique	E	very high selectivity, effectively one document per value	Accelerates sign-in, lookup, and duplicate prevention.
users	{ username: 1 } unique	E	very high selectivity, also a natural unique key	Supports profile lookup and username availability checks.
playlists	{ user: 1, _id: -1 }	E then S	user is medium-cardinality; _id gives deterministic newest-first pagination	Optimizes the authenticated playlist dashboard and owner-specific browsing.
listening_history	{ genre: 1, playedAt: -1, songId: 1 }	E then R then S	genre is low-to-medium cardinality; the date range is the real filter; songId supports grouped lookups	Covers top-songs-by-genre queries without scanning unrelated genres.
listening_history	{ userId: 1, playedAt: -1 }	E then R then S	userId is high-cardinality; playedAt limits history windows efficiently	Serves user history pages, replays, and retention jobs.

Rejected indexes

- Omit { userId: 1 } on listening_history because { userId: 1, playedAt: -1 } already serves the same access path and removes the need for a second write-time B-tree.
- Omit { likedSongs: 1 } on users because it is a multikey array with poor selectivity and would raise the write cost of every like/unlike operation.

3. Optimized Aggregation Pipelines

The pipelines below are production-oriented rather than demo snippets. They front-load selective predicates, keep the working set narrow, and defer joins until after grouping so the aggregation memory footprint stays controlled.

3.1 Top Songs by Genre (Last 30 Days)

This pipeline assumes the listening_history event stores genre at write time. That denormalization is deliberate: it allows the hot read path to use an equality predicate on genre before touching a large date range.

```
[
  { $match: { genre: genreParam, playedAt: { $gte: thirtyDaysAgo } } },
  { $project: { songId: 1, playedAt: 1 } },
  { $group: { _id: "$songId", plays: { $sum: 1 } } },
  { $sort: { plays: -1 } },
  { $limit: 20 },
  {
```

```

    $lookup: {
      from: "songs",
      localField: "_id",
      foreignField: "_id",
      as: "song"
    }
  },
  { $unwind: "$song" },
  {
    $project: {
      _id: 0,
      songId: { $toString: "$_id" },
      title: "$song.title",
      artist: "$song.artist",
      plays: 1
    }
  }
]

```

- **\$match Early Principle:** the genre and playedAt filter is first so MongoDB can use the compound listening_history index immediately.
- **The Projection Diet:** only songId and playedAt are retained before grouping, which reduces document width and memory pressure.
- **Memory Safety:** the expensive \$lookup happens only after \$group and \$limit, and the aggregation should be executed with allowDiskUse: true for predictable spill behavior when the 100MB in-memory threshold is crossed.

3.2 Monthly User Growth Analytics

This pipeline uses the users collection and assumes timestamps are available. If the current schema stays unchanged, timestamps should be enabled so growth analytics can be generated without an external reporting table.

```

[
  { $match: { createdAt: { $gte: firstDayOfWindow } } },
  { $project: { month: { $dateTrunc: { date: "$createdAt", unit: "month" } } } },
  { $group: { _id: "$month", newUsers: { $sum: 1 } } },
  { $sort: { _id: 1 } },
  { $project: { _id: 0, month: "$_id", newUsers: 1 } }
]

```

- **\$match Early Principle:** the date window is applied before any grouping so old user rows never enter the working set.
- **The Projection Diet:** only the timestamp needed for bucketing is retained, which keeps the pipeline narrow and cache-friendly.
- **Memory Safety:** the pipeline groups on a bounded month dimension instead of an unbounded per-document key, so its peak memory remains stable even when the user base grows.

4. Performance Validation

The table below simulates the explain() outcome for the most expensive read path: the top-songs-by-genre analytics query. The target outcome is not a magical zero-doc scan; it is a dramatic reduction in scanned documents, no collection scan, and a much smaller gap between work performed and data returned.

Metric	Before	After	Interpretation
Winning plan	COLLSCAN + blocking sort	IXSCAN on listening_history.{ genre, playedAt, songId }	The indexed path removes the need to read every event row.
totalDocsExamined	4,800,000	62,000	The planner now only touches the targeted genre and date window instead of the whole collection.
nReturned	20	20	The result size stays constant; the index reduces the cost of finding those 20 rows.
Execution time	~4.1 s	~150 ms	The latency drop is large enough to be visible in user-facing dashboards.
Memory behavior	Blocking stages spill pressure	No early spill; grouping occurs on a reduced set	The pipeline remains within predictable limits when allowDiskUse is enabled.

In explain() terms, a healthy index is one where totalDocsExamined collapses toward the true filtered subset rather than the full collection. For equality-driven lookups, the ideal ratio of totalDocsExamined to nReturned is close to 1. For aggregation queries, the ratio can still be higher because many raw events collapse into a small result set, but the key test is whether the plan uses IXSCAN instead of COLLSCAN and whether the examined set is proportional to the intended time window, not the entire dataset.

5. Production Readiness

The API contract below matches the current analytics route shape while still being strict enough for production documentation. It is written as an OpenAPI fragment for the /api/analytics/top-users endpoint, which is the analytics surface currently present in the repository.

```
/api/analytics/top-users:
  get:
    summary: Get the top users by playlist count
    tags:
      - Analytics
    parameters:
      - in: query
        name: limit
        required: false
        schema:
          type: integer
          default: 5
          minimum: 1
          maximum: 50
    responses:
      '200':
        description: Top users returned successfully
        content:
          application/json:
            schema:
              type: object
            properties:
              success:
                type: boolean
                example: true
              data:
```

```
      type: array
      items:
        type: object
        properties:
          userId:
            type: string
          playlistCount:
            type: integer
'500':
  description: Internal server error
```

Definition of done

- Error handling must be centralized through `asyncHandler` and the global error middleware so analytics failures never leak raw stack traces.
- Repository/service separation must remain intact: controllers handle transport, services own the aggregate pipelines, and database logic must not be embedded directly in route handlers.
- Testing must cover both happy-path responses and regression cases for query shape, pagination, and analytics output so index changes do not ship silently.

Closing note: this blueprint intentionally reduces the number of indexes while increasing the amount of work each index can satisfy. That tradeoff is what makes the system viable for 5M users: fewer write penalties, smaller memory footprint, and faster analytical reads.